

## Raport o stanie projektu

## IDUB\_2026

Głębokie uczenie do klasyfikacji i rekonstrukcji

kwantowych stanów optycznych

Audyt kodu i plan działań

Repozytorium: [https://github.com/e-mcQuantum-AI/IDUB\\_2026](https://github.com/e-mcQuantum-AI/IDUB_2026)

28 marca 2026

## Spis treści

|          |                                                               |          |
|----------|---------------------------------------------------------------|----------|
| <b>1</b> | <b>Opis projektu</b>                                          | <b>4</b> |
| <b>2</b> | <b>Wprowadzenie do pojęć — dla początkujących</b>             | <b>4</b> |
| 2.1      | Pojęcia z fizyki kwantowej . . . . .                          | 4        |
| 2.1.1    | Stan kwantowy i notacja Diraca . . . . .                      | 4        |
| 2.1.2    | Stan czysty a stan mieszany . . . . .                         | 4        |
| 2.1.3    | Macierz gęstości . . . . .                                    | 5        |
| 2.1.4    | Przestrzeń Hilberta i cutoff . . . . .                        | 5        |
| 2.1.5    | Operatory kreacji i anihilacji . . . . .                      | 5        |
| 2.1.6    | Funkcja Wignera . . . . .                                     | 6        |
| 2.1.7    | Operator przesunięcia . . . . .                               | 6        |
| 2.1.8    | Równanie Lindblada i operatory kolapsu . . . . .              | 6        |
| 2.2      | Stany kwantowe zaimplementowane w projekcie . . . . .         | 8        |
| 2.3      | Kanały szumu kwantowego . . . . .                             | 8        |
| 2.4      | Pojęcia programistyczne . . . . .                             | 8        |
| <b>3</b> | <b>Ocena obecnego stanu</b>                                   | <b>9</b> |
| 3.1      | Architektura — ocena: DOBRA . . . . .                         | 9        |
| 3.2      | Poprawność fizyczna — ocena: DOBRA z zastrzeżeniami . . . . . | 9        |
| 3.3      | Jakość kodu — ocena: PRZECIĘTNA . . . . .                     | 10       |

|           |                                                                           |           |
|-----------|---------------------------------------------------------------------------|-----------|
| <b>4</b>  | <b>Wykryte błędy i problemy</b>                                           | <b>10</b> |
| 4.1       | Błędy krytyczne . . . . .                                                 | 10        |
| 4.2       | Problemy istotne . . . . .                                                | 11        |
| 4.3       | Problemy mniejsze . . . . .                                               | 11        |
| <b>5</b>  | <b>Kluczowe fragmenty kodu źródłowego</b>                                 | <b>11</b> |
| <b>6</b>  | <b>Plan kolejnych działań — podsumowanie faz</b>                          | <b>13</b> |
| <b>7</b>  | <b>Faza 1 — Naprawa błędów krytycznych</b>                                | <b>13</b> |
| 7.1       | Dodanie plików <code>__init__.py</code> . . . . .                         | 13        |
| 7.2       | Zmiana nazwy <code>binomal.py</code> → <code>binomial.py</code> . . . . . | 14        |
| 7.3       | Dodanie dziedziczenia w kanałach szumu . . . . .                          | 15        |
| 7.4       | Naprawa generowania stanu dwumianowego . . . . .                          | 15        |
| 7.5       | Weryfikacja wzorów matematycznych . . . . .                               | 15        |
| 7.5.1     | Stan dwumianowy . . . . .                                                 | 15        |
| 7.5.2     | Stan GKP . . . . .                                                        | 16        |
| <b>8</b>  | <b>Faza 2 — Infrastruktura projektu</b>                                   | <b>16</b> |
| 8.1       | Utworzenie <code>requirements.txt</code> . . . . .                        | 16        |
| 8.2       | Parametryzacja ścieżek i stałych . . . . .                                | 16        |
| 8.3       | Ujednolicenie języka . . . . .                                            | 17        |
| 8.4       | Rozbudowa <code>README.md</code> . . . . .                                | 17        |
| <b>9</b>  | <b>Faza 3 — Jakość kodu</b>                                               | <b>17</b> |
| 9.1       | Adnotacje typów . . . . .                                                 | 17        |
| 9.2       | Docstringi . . . . .                                                      | 18        |
| 9.3       | Walidacja danych wejściowych . . . . .                                    | 18        |
| 9.4       | Obsługa błędów w skryptach . . . . .                                      | 19        |
| 9.5       | Referencje do literatury . . . . .                                        | 19        |
| <b>10</b> | <b>Faza 4 — Testy</b>                                                     | <b>19</b> |
| 10.1      | Konfiguracja <code>pytest</code> . . . . .                                | 19        |
| 10.2      | Testy jednostkowe stanów . . . . .                                        | 20        |
| 10.3      | Testy kanałów szumu . . . . .                                             | 21        |
| 10.4      | Testy walidacji . . . . .                                                 | 22        |
| 10.5      | Testy integracyjne pipeline . . . . .                                     | 22        |
| <b>11</b> | <b>Faza 5 — Rozwój funkcjonalności</b>                                    | <b>23</b> |
| 11.1      | Nowe kanały szumu . . . . .                                               | 23        |
| 11.2      | Generowanie stanów mieszanych . . . . .                                   | 23        |

---

|                                                  |           |
|--------------------------------------------------|-----------|
| 11.3 Konfiguracja CI/CD . . . . .                | 23        |
| 11.4 Moduł ML — klasyfikator stanów . . . . .    | 24        |
| 11.5 Optymalizacja wydajności . . . . .          | 24        |
| <b>12 Harmonogram i zależności między fazami</b> | <b>24</b> |
| <b>13 Diagramy architektury</b>                  | <b>25</b> |
| <b>14 Podsumowanie</b>                           | <b>26</b> |

## 1. Opis projektu

Projekt ma na celu stworzenie narzędzia do generowania syntetycznych danych obrazowych (funkcji Wignera) kwantowych stanów optycznych, które posłużą jako zbiór treninowy do klasyfikacji i rekonstrukcji stanów za pomocą głębokiego uczenia.

Obecny kod obejmuje:

- Implementację 7 stanów kwantowych (Fock, koherentny, kot Schrödingera, GKP, dwumianowy, termiczny, próżnia),
- Kanały szumu kwantowego (straty fotonowe, mieszanina stanów),
- Pomiar funkcją Wignera z potokiem pomiarowym (pipeline),
- Skrypty do generowania i dzielenia obrazów zbioru danych.

**Rozmiar kodu:** ok. 330 linii Pythona w 18 plikach.

## 2. Wprowadzenie do pojęć — dla początkujących

Ten rozdział wyjaśnia kluczowe pojęcia z fizyki kwantowej i programowania, które pojawiają się w dalszej części raportu.

### 2.1. Pojęcia z fizyki kwantowej

#### 2.1.1. Stan kwantowy i notacja Diraca

**Stan kwantowy** to matematyczny opis układu kwantowego. W tym projekcie opisujemy stany światła (fotony w jednym modzie pola elektromagnetycznego).

**Ket**  $|\psi\rangle$  (czytaj: *ket psi*) — wektor opisujący stan kwantowy w notacji Diraca. Na przykład  $|3\rangle$  oznacza stan z dokładnie 3 fotonami. W kodzie ket to wektor kolumnowy (obiekt `Qobj` z biblioteki `QuTiP`).

**Bra**  $\langle\psi|$  — wektor sprzężony do ketu (transponowany i zespolenie sprzężony, czyli hermitowsko sprzężony). Potrzebny do obliczeń, np. iloczynu skalarnego  $\langle\psi|\psi\rangle$ .

**Iloczyn skalarny**  $\langle\psi|\psi\rangle$  — miara „nakładania się” dwóch stanów. Jeśli  $\langle\psi|\psi\rangle = 1$ , stan jest **znormalizowany** (ma sens fizyczny).

#### 2.1.2. Stan czysty a stan mieszany

- **Stan czysty** — układ jest w jednym, określonym stanie kwantowym  $|\psi\rangle$ . Przykład: dokładnie 3 fotony.

- **Stan mieszany** — układ jest w jednym z kilku stanów z pewnym prawdopodobieństwem (mieszanina statystyczna). Przykład: stan termiczny — nie wiemy, ile jest fotonów, znamy tylko średnią. Stanu mieszanego **nie da się** opisać pojedynczym ketem.

### 2.1.3. Macierz gęstości

**Macierz gęstości**  $\rho$  — uniwersalny sposób opisu stanu kwantowego, działa zarówno dla stanów czystych, jak i mieszanych:

$$\text{Stan czysty: } \rho = |\psi\rangle \langle\psi| \quad (1)$$

$$\text{Stan mieszany: } \rho = \sum_i p_i |\psi_i\rangle \langle\psi_i| \quad (2)$$

W kodzie: `state.density_matrix()` zwraca macierz kwadratową.

#### Fizyczność macierzy gęstości

Poprawna macierz gęstości musi spełniać 3 warunki:

1. **Hermitowskość:**  $\rho = \rho^\dagger$  — macierz jest równa swojemu sprzężeniu hermitowskiemu (tzn. transpozycji ze sprzężeniem zespolonym).
2. **Ślad = 1:**  $\text{Tr}(\rho) = 1$  — prawdopodobieństwa sumują się do 1.
3. **Dodatnia półokreśloność:** wszystkie wartości własne  $\geq 0$  — prawdopodobieństwa nie mogą być ujemne.

### 2.1.4. Przestrzeń Hilberta i cutoff

**Przestrzeń Hilberta** to przestrzeń matematyczna, w której „żyją” stany kwantowe. Jej wymiar jest teoretycznie nieskończony (oscylator może zawierać dowolnie wiele fotonów), ale w komputerze musimy ją **obciąć** — parametr `cutoff` określa, ile stanów bazowych ( $0, 1, 2, \dots, \text{cutoff} - 1$  fotonów) uwzględniamy. Na przykład `cutoff=32` oznacza, że rozważamy stany od 0 do 31 fotonów.

### 2.1.5. Operatory kreacji i anihilacji

Operatory matematyczne na przestrzeni Hilberta:

$$\hat{a} |n\rangle = \sqrt{n} |n-1\rangle \quad \text{anihilacja — „zabiera” foton} \quad (3)$$

$$\hat{a}^\dagger |n\rangle = \sqrt{n+1} |n+1\rangle \quad \text{kreacja — „dodaje” foton} \quad (4)$$

$$\hat{n} = \hat{a}^\dagger \hat{a} \quad \text{operator liczby fotonów: } \hat{n} |n\rangle = n |n\rangle \quad (5)$$

W kodzie: `destroy()` =  $\hat{a}$ , `create()` =  $\hat{a}^\dagger$ .

### 2.1.6. Funkcja Wignera

Sposób wizualizacji stanu kwantowego jako „mapy cieplnej” w **przestrzeni fazowej** (oś  $x$  = pozycja  $q$ , oś  $y$  = pęd  $p$ ). Formalnie, dla macierzy gęstości  $\rho$  (w jednostkach, w których  $\hbar = 2$ , zgodnie z konwencją QuTiP):

$$W(q, p) = \frac{1}{\pi\hbar} \int_{-\infty}^{\infty} \langle q + y | \rho | q - y \rangle e^{2ipy/\hbar} dy \quad (6)$$

gdzie  $\hbar$  to zredukowana stała Plancka (podstawowa stała fizyczna łącząca skalę kwantową z klasyczną). W praktyce nie trzeba liczyć tej całki ręcznie — biblioteka QuTiP oblicza ją automatycznie funkcją `wigner(rho, xvec, xvec)`.

Każdy typ stanu ma charakterystyczny wzór na wykresie Wignera — to właśnie te wzory projekt chce klasyfikować za pomocą uczenia maszynowego. Funkcja Wignera może przyjmować **wartości ujemne** — to cecha typowo kwantowa, niemożliwa w fizyce klasycznej (ujemność Wignera jest świadectwem nieklasyczności stanu). Schemat potoku pomiarowego przedstawia Rysunek 3.

### 2.1.7. Operator przesunięcia

$\hat{D}(\alpha)$  — przesuwa stan w przestrzeni fazowej o wektor  $\alpha$  (liczba zespolona:  $\text{Re}(\alpha)$  = przesunięcie w  $q$ ,  $\text{Im}(\alpha)$  = przesunięcie w  $p$ ). W kodzie: `displace(cutoff, alpha)`.

### 2.1.8. Równanie Lindblada i operatory kolapsu

**Równanie Lindblada** (zwane też *master equation*) [1] — równanie opisujące ewolucję czasową stanu kwantowego oddziałującego z otoczeniem (układ otwarty). W tym projekcie służy do symulacji szumu: zamiast izolowanej ewolucji unitarnej, stan  $\rho$  ewoluuje z uwzględnieniem strat do otoczenia.

**Operator kolapsu**  $\hat{c}$  — operator opisujący konkretny mechanizm oddziaływania z otoczeniem w równaniu Lindblada. Różne operatory kolapsu modelują różne rodzaje szumu:

- $\hat{c} = \sqrt{\gamma} \hat{a}$  — straty fotonowe (fotony uciekają z układu),
- $\hat{c} = \sqrt{\gamma} \hat{a}^\dagger \hat{a}$  — defazowanie (utrata informacji o fazie),
- $\hat{c} = \sqrt{\gamma} \hat{a}^\dagger$  — spontaniczne wzmocnienie (dodawanie fotonów).

W kodzie: parametr `c_ops` w funkcji `mesolve()` z biblioteki QuTiP [2].

Tabela 1: Stany kwantowe zaimplementowane w projekcie.

| Stan       | Symbol                             | Opis intuicyjny                                                               | Przykład                                 |
|------------|------------------------------------|-------------------------------------------------------------------------------|------------------------------------------|
| Próżnia    | $ 0\rangle$                        | Brak fotonów, lecz niezerowe fluktuacje kwantowe (energia punktu zerowego).   | Najniższy stan energetyczny pola         |
| Fock       | $ n\rangle$                        | Dokładnie $n$ fotonów. Stan o określonej liczbie cząstek.                     | Foton z emitera kwantowego               |
| Koherentny | $ \alpha\rangle$                   | Stan najbliższy klasycznemu światłu. $\langle \hat{n} \rangle =  \alpha ^2$ . | Światło lasera                           |
| Kot        | $ \alpha\rangle +  -\alpha\rangle$ | Superpozycja dwóch stanów koherentnych — „jednocześnie tu i tam”.             | Eksperyment Schrödingera                 |
| Termiczny  | $\rho_{\text{th}}$                 | Stan mieszany w równowadze termicznej (rozkład Bosego–Einsteina).             | Promieniowanie zarówki                   |
| Dwumianowy | $ \psi_{\text{bin}}\rangle$        | Superpozycja stanów Focka ze współczynnikami dwumianowymi [3].                | Pamięci kwantowe                         |
| GKP        | $ \text{GKP}\rangle$               | Kod korekcji błędów: kubit zakodowany w oscylatorze harmonicznym [4].         | Komputery kwantowe ze zmiennymi ciągłymi |

## 2.2. Stany kwantowe zaimplementowane w projekcie

## 2.3. Kanały szumu kwantowego

Kanał szumu to operacja symulująca niedoskonałości fizyczne — wprowadza szum do stanu kwantowego. Zaimplementowane w projekcie kanały (straty, mieszanina) zmieniają stan w sposób nieodwracalny:

Tabela 2: Kanały szumu zaimplementowane w projekcie.

| Kanał           | Co robi                                                                                                                              | Analogia                   |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| Straty fotonowe | Fotony „uciekają” z układu z intensywnością $\gamma$ (im większe $\gamma$ , tym więcej strat). Zmniejsza $\langle \hat{n} \rangle$ . | Światło przez brudne szkło |
| Mieszanina      | Miesza dwa stany: $\rho' = p\rho + (1 - p)\rho_{\text{other}}$ .                                                                     | Niedoskonały detektor      |

## 2.4. Pojęcia programistyczne

**\_\_init\_\_.py** Specjalny plik w Pythonie, który mówi interpreterowi: „ten katalog jest pakietem Pythona”. Bez niego nie można pisać `from src.physics import FockState`.

**Klasa abstrakcyjna (ABC)** Klasa definiująca „umowę” (interfejs): mówi, jakie metody muszą mieć klasy potomne, ale sama ich nie implementuje. Np. `QuantumState` wymaga metody `ket()`.

**Dziedziczenie** Mechanizm, w którym klasa potomna przejmuję cechy klasy nadrzędnej. `class LossChannel(QuantumChannel)` oznacza: „`LossChannel` jest rodzajem `QuantumChannel`”.

**Type hints** Adnotacje typów: `def ket(self) -> Qobj`: mówi „ta metoda zwraca `Qobj`”. Python ich nie wymusza, ale pomagają w wykrywaniu błędów.

**Docstring** Komentarz dokumentacyjny w potrójnych cudzysłowach `"""`. Opisuje co robi funkcja/klasa.

**requirements.txt** Plik z listą bibliotek (np. `qutip>=5.0`). Instalacja: `pip install -r requirements.txt`.

**pytest / fixture** Framework do testów automatycznych. Fixture to przygotowane dane współdzielone przez testy.

**Pipeline** Wzorzec „potok”: dane przechodzą kolejno przez etapy. Tu: stan  $\rightarrow$  szum  $\rightarrow$  pomiar Wignera  $\rightarrow$  obraz.



CI/CD Automatyczne uruchamianie testów przy każdym `git push`.

### 3. Ocena obecnego stanu

#### 3.1. Architektura — ocena: DOBRA

Projekt ma przejrzystą strukturę obiektową z użyciem klas abstrakcyjnych (ABC):

```

1 src/physics/
2 |-- state/          -- implementacje stanów kwantowych
3 |-- noise/          -- kanały szumowe
4 |-- measurement/    -- pomiar funkcja Wignera + pipeline
5 |-- divider.py      -- narzędzie do dzielenia obrazów
6 |-- generate.py      -- skrypt generujący zbiór danych
7 |-- hilbert.py       -- operatory przestrzeni Hilberta
8 '-- validation.py   -- walidacja stanów fizycznych

```

Listing 1: Struktura katalogów projektu.

Dobór bibliotek jest trafny: **QuTiP** [2], NumPy, SciPy, Matplotlib, Pillow (biblioteka do przetwarzania obrazów, importowana jako PIL).

Zależności klas przedstawia Rysunek 2.

#### 3.2. Poprawność fizyczna — ocena: DOBRA z zastrzeżeniami

Tabela 3: Poprawność implementacji stanów kwantowych.

| Stan                                    | Implementacja                             | Poprawność |
|-----------------------------------------|-------------------------------------------|------------|
| Próżnia ( <code>vacuum.py</code> )      | <code>basis(N, 0)</code>                  | ✓          |
| Fock ( <code>fock.py</code> )           | <code>basis(N, n)</code>                  | ✓          |
| Koherentny ( <code>coherent.py</code> ) | <code>QuTiP coherent()</code>             | ✓          |
| Kot ( <code>cat.py</code> )             | superpozycja stanów koherentnych          | ✓          |
| Termiczny ( <code>thermal.py</code> )   | <code>thermal_dm()</code> , stan mieszany | ✓          |
| GKP ( <code>gkp.py</code> )             | obwiednia gaussowska + przesunięcia       | △          |
| Dwumianowy ( <code>binomal.py</code> )  | współczynniki dwumianowe                  | △          |

✓ = poprawna, △ = wymaga weryfikacji

Tabela 4: Ocena aspektów jakości kodu.

| Aspekt              | Ocena | Uwagi                      |
|---------------------|-------|----------------------------|
| Struktura katalogów | Dobra | Logiczny podział na moduły |
| Czytelność          | Dobra | Krótkie, jasne funkcje     |
| Styl (PEP8)         | Dobra | Generalnie zgodny          |
| Dokumentacja        | Słaba | Brak docstringów           |
| Typowanie           | Brak  | Zero adnotacji typów       |
| Obsługa błędów      | Brak  | Brak walidacji wejścia     |
| Testy               | Brak  | Zero testów                |
| Konfiguracja        | Słaba | Ścieżki na sztywno         |

### 3.3. Jakość kodu — ocena: PRZECIĘTNA

## 4. Wykryte błędy i problemy

### 4.1. Błędy krytyczne

#### B1. Brak plików `__init__.py`

Żaden pakiet nie ma pliku `__init__.py`. Import modułów przez `from src.physics` `import ...` jest niemożliwy. Dotyczy: `src/`, `src/physics/`, `src/physics/state/`, `src/physics/noise/`, `src/physics/measurement/`.

#### B2. Błąd matematyczny w `generate.py` (linia 44)

Kod tworzy superpozycję dwóch stanów Focka zamiast prawdziwego stanu dwumianowego:

```
state_binomial = (fock(N, n) + p * fock(N, n+1)).unit()
```

To NIE jest stan dwumianowy — to  $|n\rangle + p|n+1\rangle$ . Prawdziwa implementacja istnieje w `state/binomial.py`, ale nie jest używana przez generator.

#### B3. Literówka w nazwie pliku

Plik `state/binomal.py` powinien nazywać się `binomial.py`.

#### B4. Kanały szumu nie dziedziczą po klasie bazowej

`LossChannel` i `MixtureChannel` nie dziedziczą po `QuantumChannel` (zdefiniowanej w `noise/base.py`). Oznacza to, że Python nie wie, iż te klasy są kanałami kwantowymi — nie można ich używać zamiennie w kodzie oczekującym obiektu typu `QuantumChannel` (np. w `MeasurementPipeline`).

## 4.2. Problemy istotne

**P1. Ścieżki wpisane na sztywno** — `divider.py`: `'quantum_dataset/clean/'`, `generate.py`: `'quantum_dataset/'`. Brak przenośności.

**P2. Brak requirements.txt** — Zależności (QuTiP, NumPy, SciPy, Matplotlib, Pillow, OpenCV) nie są udokumentowane.

**P3. Brak walidacji danych wejściowych** — `BinomialState`: brak sprawdzenia  $p \in [0, 1]$ ,  $N \geq 0$ ; `MixtureChannel`: brak sprawdzenia  $p \in [0, 1]$ ; `LossChannel`: brak sprawdzenia  $\gamma \geq 0$ .

**P4. Wzór GKP nieudokumentowany** — Stan GKP (Gottesman–Kitaev–Preskill) [4] to kwantowy kod korekcji błędów kodujący kubit w oscylatorze harmonicznym. Idealny stan GKP to nieskończony grzebień delt Diraca w przestrzeni pozycji, rozmieszczonych co  $2\sqrt{\pi}$ . W praktyce stosuje się przybliżenie z obwiednią gaussowską o szerokości  $\delta$ . Brak referencji w kodzie utrudnia weryfikację:

```
weight = np.exp(-(2 * s * np.sqrt(np.pi))**2 / (2 * self.delta**2))
```

Listing 2: Nieudokumentowany wzór GKP w `gkp.py`:25.

**P5. Mieszanie języków** — Komentarze w `generate.py` po polsku, reszta kodu po angielsku.

## 4.3. Problemy mniejsze

- **P6.** Stała magiczna `-1e-10` w `validation.py` — powinna być konfigurowalna.
- **P7.** Parametr `tlist=[0, 1]` w `loss.py` — brak wyjaśnienia jednostek czasu.
- **P8.** Brak logowania — tylko `print()` w `generate.py`.

## 5. Kluczowe fragmenty kodu źródłowego

```
1 from abc import ABC, abstractmethod
2 from qutip import Qobj
3
4 class QuantumState(ABC):
5
6     @abstractmethod
7     def ket(self) -> Qobj:
8         pass
9
10    def density_matrix(self) -> Qobj:
```

```

11     psi = self.ket()
12     return psi * psi.dag()

```

Listing 3: Klasa bazowa stanów kwantowych (state/base.py).

```

1 from abc import ABC, abstractmethod
2 from qutip import Qobj
3
4 class QuantumChannel(ABC):
5
6     @abstractmethod
7     def apply(self, rho: Qobj) -> Qobj:
8         pass

```

Listing 4: Klasa bazowa kanałów kwantowych (noise/base.py).

```

1 import numpy as np
2 from qutip import displace, basis
3 from .base import QuantumState
4
5 class GKPState(QuantumState):
6
7     def __init__(self, cutoff: int, delta: float = 0.3, grid_size:
8         int = 5):
9         self.cutoff = cutoff
10        self.delta = delta
11        self.grid_size = grid_size
12
13    def ket(self):
14        psi = 0
15        vacuum = basis(self.cutoff, 0)
16
17        for s in range(-self.grid_size, self.grid_size + 1):
18            q_shift = 2 * s * np.sqrt(np.pi)
19            D = displace(self.cutoff, q_shift / np.sqrt(2))
20            weight = np.exp(-(2 * s * np.sqrt(np.pi))**2
21                            / (2 * self.delta**2))
22            psi += weight * D * vacuum
23
24    return psi.unit()

```

Listing 5: Implementacja stanu GKP (state/gkp.py).

```

1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from qutip import fock, coherent, wigner, ket2dm
5
6 def generate_dataset(n_samples=10, resolution=100):
7     paths = setup_folders()
8     N = 32
9     xvec = np.linspace(-5, 5, resolution)
10
11     for i in range(n_samples):
12         # --- FOCK STATE ---
13         n_photons = np.random.randint(0, 8)
14         state_fock = fock(N, n_photons)
15         w_clean = wigner(state_fock, xvec, xvec)
16         plt.imsave(os.path.join(paths["clean"],
17                                 f"fock_n{n_photons}_id{i}.png"), w_clean, cmap='RdBu_r',
18                     )
19
20         # --- BINOMIAL STATE (BLAD! To nie jest stan dwumianowy)
21         # ---
22         n = np.random.randint(0, 8)
23         p = np.random.uniform(0.1, 0.9)
24         state_binomial = (fock(N, n) + p * fock(N, n+1)).unit() #
25         # B2!

```

Listing 6: Generator zbioru danych (generate.py) — fragment z błędem B2.

## 6. Plan kolejnych działań — podsumowanie faz

## 7. Faza 1 — Naprawa błędów krytycznych

### 7.1. Dodanie plików \_\_init\_\_.py

**Problem:** Brak plików \_\_init\_\_.py uniemożliwia import pakietów Pythona.

**Kroki:**

1. Utworzyć puste pliki \_\_init\_\_.py w katalogach: src/, src/physics/, src/physics/state/, src/physics/noise/, src/physics/measurement/.
2. W src/physics/\_\_init\_\_.py dodać eksporty głównych klas (Listing 7).

Tabela 5: Podsumowanie faz planu działań.

| Faza | Priorytet | Zakres                                                                   | Czas    |
|------|-----------|--------------------------------------------------------------------------|---------|
| 1    | WYSOKI    | Naprawa błędów krytycznych (B1–B4, weryfikacja wzorów)                   | ~4h     |
| 2    | WYSOKI    | Infrastruktura ( <code>requirements.txt</code> , parametryzacja, README) | ~5h     |
| 3    | ŚREDNI    | Jakość kodu (typy, docstringi, walidacja, referencje)                    | ~11h    |
| 4    | ŚREDNI    | Testy (pytest, testy stanów, kanałów, pipeline)                          | ~10h    |
| 5    | NISKI     | Rozwój (nowe kanały, CI/CD, moduł ML)                                    | otwarte |

3. Poprawić niespójne importy — w `coherent.py` i `vacuum.py` jest import bezwzględny (`from src.physics.state.base`), a w pozostałych plikach względny (`from .base`). Ujednolicić do importów względnych.

4. Sprawdzić: `python -c "from src.physics import FockState"`.

```

1 from .state.fock import FockState
2 from .state.coherent import CoherentState
3 from .state.cat import CatState
4 from .state.vacuum import VacuumState
5 from .state.thermal import ThermalState
6 from .state.gkp import GKPState
7 from .state.binomial import BinomialState
8 from .noise.loss import LossChannel
9 from .noise.mixture import MixtureChannel
10 from .measurement.wigner import WignerMeasurement
11 from .measurement.pipeline import MeasurementPipeline

```

Listing 7: Proponowana zawartość `src/physics/__init__.py`.

**Kryterium zakończenia:** Polecenie `python -c "from src.physics import FockState"` wykonuje się bez błędów.

## 7.2. Zmiana nazwy `binomal.py` → `binomial.py`

1. `git mv src/physics/state/binomal.py src/physics/state/binomial.py`
2. Zaktualizować wszystkie importy.
3. `grep -r "binomal"` — upewnić się, że nie ma starych odwołań.

### 7.3. Dodanie dziedziczenia w kanałach szumu

```

1 # PRZED:
2 class LossChannel:
3 # PO:
4 from .base import QuantumChannel
5 class LossChannel(QuantumChannel):
6
7 # PRZED:
8 class MixtureChannel:
9 # PO:
10 from .base import QuantumChannel
11 class MixtureChannel(QuantumChannel):

```

Listing 8: Naprawa dziedziczenia w noise/loss.py i noise/mixture.py.

**Kryterium:** `isinstance(LossChannel(32, 0.1), QuantumChannel)` zwraca `True`.

### 7.4. Naprawa generowania stanu dwumianowego

```

1 # PRZED (linia 44):
2 state_binomial = (fock(N, n) + p * fock(N, n+1)).unit()
3
4 # PO:
5 from src.physics.state.binomial import BinomialState
6 state_binomial = BinomialState(N=n, p=p, cutoff=N).ket()

```

Listing 9: Naprawa błędu B2 w generate.py.

### 7.5. Weryfikacja wzorów matematycznych

#### 7.5.1. Stan dwumianowy

Obecna implementacja:

$$|\psi_{\text{bin}}\rangle = \sum_{n=0}^N \sqrt{\binom{N}{n}} p^{n/2} (1-p)^{(N-n)/2} |n\rangle \quad (7)$$

Kroki weryfikacji:

1. Sprawdzić definicję w Michael et al. [3].

2. Potwierdzić normalizację:  $\langle \psi | \psi \rangle = \sum \binom{N}{n} p^n (1-p)^{N-n} = 1$  (wzór dwumianowy Newtona).
3. Upewnić się, że `cutoff > N`.

### 7.5.2. Stan GKP

Idealny stan GKP to nieskończony grzebień delt Diraca [4]:

$$|0_{\text{GKP}}\rangle \propto \sum_s \delta(q - 2s\sqrt{\pi}) \quad (8)$$

$$|1_{\text{GKP}}\rangle \propto \sum_s \delta(q - (2s+1)\sqrt{\pi}) \quad (9)$$

Przybliżenie skończonej energii zastosowane w projekcie:

$$|\text{GKP}\rangle \propto \sum_s \exp\left(-\frac{(2s\sqrt{\pi})^2}{2\delta^2}\right) \hat{D}\left(\frac{2s\sqrt{\pi}}{\sqrt{2}}\right) |0\rangle \quad (10)$$

Parametr  $\delta$  kontroluje kompromis: mniejszy  $\delta \rightarrow$  bliżej ideału, ale większa wymagana energia. Na wykresie Wignera stan GKP wyróżnia się charakterystyczną **regularną siatką pików** (patrz Rysunek 4).

**Kroki weryfikacji:** Porównać z rów. (18) w [4]. Sprawdzić konwencję QuTiP:  $\hat{D}(\alpha)$  przesuwa o  $\text{Re}(\alpha)$  w  $q$ .

## 8. Faza 2 — Infrastruktura projektu

### 8.1. Utworzenie requirements.txt

```
1 qutip>=5.0
2 numpy>=1.24
3 scipy>=1.10
4 matplotlib>=3.7
5 Pillow>=10.0
```

Listing 10: Proponowany requirements.txt.

Opcjonalnie requirements-dev.txt z pytest>=7.0 i opencv-python>=4.8.

### 8.2. Parametryzacja ścieżek i stałych

```
1 from dataclasses import dataclass
2 from pathlib import Path
```



```

3
4 @dataclass
5 class GeneratorConfig:
6     output_dir: Path = Path("quantum_dataset")
7     cutoff: int = 32
8     resolution: int = 100
9     x_range: float = 5.0
10    n_samples: int = 10
11    cmap: str = "RdBu_r"

```

Listing 11: Proponowany plik konfiguracyjny config.py.

Dodać interfejs CLI z argparse: `python generate.py -n-samples 100 -resolution 200`.

### 8.3. Ujednolicenie języka

Przetłumaczyć komentarze polskie na angielski (m.in. w `generate.py`, `gkp.py`).

Kryterium: `grep -rn "[ąęłńóśż]"src/` nie zwraca wyników.

### 8.4. Rozbudowa README.md

Dodać sekcje: opis projektu, struktura, wymagania, instalacja, przykłady użycia, autorzy. Przykład szybkiego startu:

```

1 from src.physics.state.fock import FockState
2 from src.physics.measurement.wigner import WignerMeasurement
3
4 state = FockState(n=3, cutoff=32)
5 wm = WignerMeasurement(x_max=5, resolution=100)
6 W = wm.measure(state.density_matrix())

```

Listing 12: Przykład użycia do umieszczenia w README.

## 9. Faza 3 — Jakość kodu

### 9.1. Adnotacje typów

```

1 class FockState(QuantumState):
2     def __init__(self, n: int, cutoff: int) -> None:
3         self.n: int = n
4         self.cutoff: int = cutoff

```

```

5
6 def ket(self) -> Qobj:
7     return basis(self.cutoff, self.n)

```

Listing 13: Przykład adnotacji typów dla FockState.

Kryterium: `mypy src/` przechodzi bez błędów krytycznych (`mypy` to narzędzie do statycznego sprawdzania poprawności typów w Pythonie — wykrywa błędy typów bez uruchamiania kodu).

## 9.2. Docstringi

```

1 class GKPState(QuantumState):
2     """Approximate Gottesman-Kitaev-Preskill (GKP) state.
3
4     GKP state encodes a logical qubit in a harmonic oscillator,
5     protecting against small displacement errors in position (q)
6     and momentum (p). The ideal state is an infinite comb of
7     Dirac deltas spaced by 2*sqrt(pi). This implementation uses
8     a finite-energy approximation with a Gaussian envelope of
9     width delta:
10
11     |GKP> ~ sum_s exp(-2*pi*s**2 / delta**2) D(s*sqrt(2*pi))
12         |0>
13
14     Reference:
15     D. Gottesman, A. Kitaev, J. Preskill,
16     "Encoding a qubit in an oscillator",
17     Phys. Rev. A 64, 012310 (2001).
18     https://doi.org/10.1103/PhysRevA.64.012310
19
20     Args:
21     cutoff: Hilbert space dimension.
22     delta: Envelope width (smaller = more ideal).
23     grid_size: Number of peaks on each side.
24 """

```

Listing 14: Przykładowy docstring z referencją do literatury (GKP).

## 9.3. Walidacja danych wejściowych

```
1 # BinomialState.__init__()
2 if N < 0:
3     raise ValueError(f"N must be non-negative, got {N}")
4 if not 0 <= p <= 1:
5     raise ValueError(f"p must be in [0, 1], got {p}")
6 if cutoff <= N:
7     raise ValueError(f"cutoff ({cutoff}) must be > N ({N})")
8
9 # LossChannel.__init__()
10 if gamma < 0:
11     raise ValueError(f"gamma must be non-negative, got {gamma}")
```

Listing 15: Przykłady walidacji parametrów.

## 9.4. Obsługa błędów w skryptach

Dodać try/except w generate.py i sprawdzanie istnienia pliku w divider.py.

## 9.5. Referencje do literatury

Dodać referencje w docstringach: GKP [4], stan dwumianowy [3], kanał strat — równanie Lindblada [1, 5].

# 10. Faza 4 — Testy

## 10.1. Konfiguracja pytest

Utworzyć katalog tests/ ze strukturą:

```
1 tests/
2 |-- __init__.py
3 |-- conftest.py          -- wspólne fixtures
4 |-- test_states.py       -- testy stanów
5 |-- test_noise.py        -- testy kanałów szumu
6 |-- test_validation.py   -- testy walidacji
7 |-- test_measurement.py  -- testy pomiarów
8 '-- test_pipeline.py     -- testy integracyjne
```

Listing 16: Struktura katalogu testów.

## 10.2. Testy jednostkowe stanów

Każdy stan kwantowy musi spełniać warunki fizyczne z sekcji 2. Tabela 6 przedstawia testy wspólne.

Tabela 6: Testy wspólne dla wszystkich stanów kwantowych.

| Test                         | Co sprawdzamy?                | Dlaczego to ważne?                           | Kod                              |
|------------------------------|-------------------------------|----------------------------------------------|----------------------------------|
| Normalizacja                 | $\langle\psi \psi\rangle = 1$ | Prawdopodobieństwa = 1                       | <code>state.ket().norm()</code>  |
| Ślad $\rho$                  | $\text{Tr}(\rho) = 1$         | Jak wyżej, dla macierzy                      | <code>rho.tr()</code>            |
| Hermitowskość                | $\rho = \rho^\dagger$         | Wielkości obserwowalne muszą być rzeczywiste | <code>rho.isherm</code>          |
| Dodatnia pół-<br>określoność | wartości własne $\geq 0$      | Prawdopodobieństwa nieujemne                 | <code>rho.eigenenergies()</code> |

Testy specyficzne:

```

1 class TestFockState:
2     def test_photon_number(self, cutoff):
3         """Stan Focka |n> powinien mieć dokładnie n fotonów.
4         n_hat = a.dag() * a to operator liczby fotonów.
5         expect(n_hat, rho) oblicza wartość oczekiwaną <n>.
6         """
7         state = FockState(n=3, cutoff=cutoff)
8         rho = state.density_matrix()
9         a = destroy(cutoff) # operator anihilacji
10        n_hat = a.dag() * a # operator liczby fotonów
11        assert abs(expect(n_hat, rho) - 3.0) < 1e-10
12
13 class TestCoherentState:
14     def test_mean_photon_number(self, cutoff):
15         """Stan koherentny |alpha> ma średnio |alpha|^2 fotonów."""
16         alpha = 2.0
17         state = CoherentState(alpha=alpha, cutoff=cutoff)
18         rho = state.density_matrix()
19         a = destroy(cutoff)
20         n_hat = a.dag() * a
21         assert abs(expect(n_hat, rho) - abs(alpha)**2) < 0.01
22

```

```

23 class TestCatState:
24     def test_symmetry(self, cutoff):
25         """Parzysty kot zawiera tylko parzyste stany Focka.
26          $|\alpha\rangle + |-\alpha\rangle \rightarrow$  tylko  $|0\rangle, |2\rangle, |4\rangle, \dots$ 
27         """
28         state = CatState(alpha=2.0, cutoff=cutoff)
29         psi = state.ket()
30         for n in [1, 3, 5]:
31             overlap = abs(basis(cutoff, n).dag() * psi)[0][0]
32             assert abs(overlap) < 1e-10
33
34 class TestThermalState:
35     def test_ket_raises(self, cutoff):
36         """Stan termiczny jest mieszany -- nie ma ketu."""
37         state = ThermalState(n_th=1.0, cutoff=cutoff)
38         with pytest.raises(NotImplementedError):
39             state.ket()

```

Listing 17: Testy stanów kwantowych — przykłady.

### 10.3. Testy kanałów szumu

#### Zasada testowania kanałów

Kanał szumu nie może złamać praw fizyki — wynik musi nadal być poprawną macierzą gęstości (hermitowska, ślad = 1, wartości własne  $\geq 0$ ).

```

1 class TestLossChannel:
2     def test_reduces_photon_number(self, cutoff):
3         """Straty zmniejszają średnią liczbę fotonów."""
4         state = FockState(n=5, cutoff=cutoff)
5         channel = LossChannel(cutoff=cutoff, gamma=0.3)
6         rho_in = state.density_matrix()
7         rho_out = channel.apply(rho_in)
8         a = destroy(cutoff)
9         n_hat = a.dag() * a
10        assert expect(n_hat, rho_out) < expect(n_hat, rho_in)
11
12    def test_zero_loss_preserves_state(self, cutoff):
13        """Brak szumu (gamma=0) = brak zmian."""
14        state = CoherentState(alpha=1.0, cutoff=cutoff)

```

```

15     channel = LossChannel(cutoff=cutoff, gamma=0.0)
16     rho_in = state.density_matrix()
17     rho_out = channel.apply(rho_in)
18     assert abs((rho_in - rho_out).norm()) < 1e-6

```

Listing 18: Testy kanałów szumu — przykłady.

## 10.4. Testy walidacji

```

1 class TestIsPhysical:
2     def test_negative_eigenvalue(self, cutoff):
3         """Macierz z ujemna wartoscia wlasna jest niefizyczna.
4         diag([1.5, -0.5, 0, ...]) -- slad=1, hermitowska,
5         ale -0.5 to ujemne prawdopodobienstwo.
6         """
7         mat = np.diag([1.5, -0.5] + [0.0] * (cutoff - 2))
8         rho_bad = Qobj(mat)
9         assert not is_physical(rho_bad)

```

Listing 19: Testy funkcji is\_physical() — wykrywanie macierzy niefizycznych.

## 10.5. Testy integracyjne pipeline

```

1 class TestMeasurementPipeline:
2     def test_all_states_produce_output(self, cutoff):
3         """Kazdy typ stanu daje poprawny obraz Wignera."""
4         wm = WignerMeasurement(x_max=5, resolution=32)
5         pipeline = MeasurementPipeline(measurement=wm)
6         states = [
7             FockState(n=1, cutoff=cutoff),
8             CoherentState(alpha=1.0, cutoff=cutoff),
9             CatState(alpha=2.0, cutoff=cutoff),
10            VacuumState(cutoff=cutoff),
11            GKPSState(cutoff=cutoff, delta=0.4),
12            BinomialState(N=3, p=0.5, cutoff=cutoff),
13        ]
14        for state in states:
15            result = pipeline.run(state)
16            assert result.shape == (32, 32)
17            assert np.isfinite(result).all()

```

Listing 20: Test integracyjny: wszystkie stany przez pipeline.

## 11. Faza 5 — Rozwój funkcjonalności

### 11.1. Nowe kanały szumu

Obecnie projekt ma 2 kanały szumu. Proponowane rozszerzenia:

**Kanał defazowania** — niszczenie informacji o fazie (kącie w przestrzeni fazowej), bez utraty fotonów. Operator kolapsu:  $\sqrt{\gamma} \hat{n}$ . Analogia: światło w ośrodku o losowo zmieniającym się współczynniku załamania.

**Kanał wzmocnienia** — dodawanie fotonów (odwrotność strat). Operator kolapsu:  $\sqrt{\gamma} \hat{a}^\dagger$ . Analogia: wzmacniacz optyczny.

**Kanał depolaryzacji** — z prawdopodobieństwem  $p$  stan zastępowany „białym szumem”:  $\rho \rightarrow (1 - p)\rho + p \cdot \mathbf{I}/d$ , gdzie  $\mathbf{I}$  to macierz jednostkowa, a  $d$  — wymiar przestrzeni Hilberta. Analogia: detektor dający losowy wynik.

### 11.2. Generowanie stanów mieszanych

Rozszerzyć generator o stany z szumem (Fock + straty, kot + defazowanie). Dodać metadane CSV.

### 11.3. Konfiguracja CI/CD

```
1 name: Tests
2 on: [push, pull_request]
3 jobs:
4   test:
5     runs-on: ubuntu-latest
6     strategy:
7       matrix:
8         python-version: ["3.10", "3.11", "3.12"]
9     steps:
10      - uses: actions/checkout@v4
11      - uses: actions/setup-python@v5
12        with:
13          python-version: ${matrix.python-version}
14      - run: pip install -r requirements-dev.txt
```

15

```
- run: pytest tests/ -v --tb=short
```

Listing 21: Proponowany `.github/workflows/tests.yml`.

## 11.4. Moduł ML — klasyfikator stanów

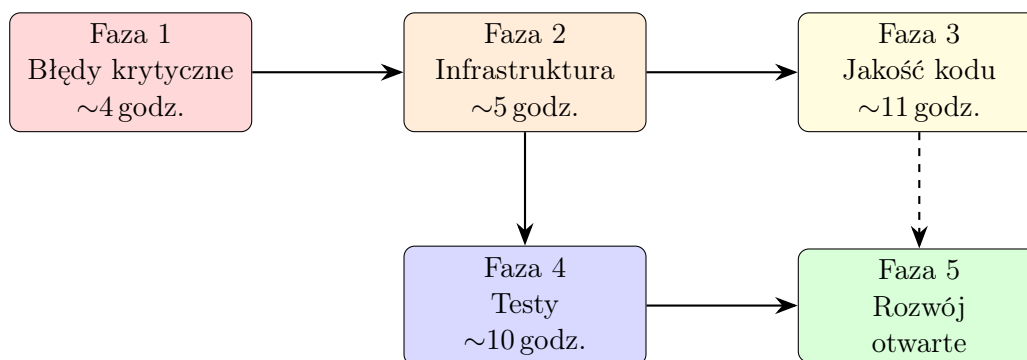
Główny cel projektu. Zadania ML:

- **Klasyfikacja:** obraz Wignera  $\rightarrow$  typ stanu (Fock / koherentny / kot / GKP / dwumianowy / termiczny),
- **Rekonstrukcja:** zaszumiony obraz  $\rightarrow$  czysty obraz,
- **Estymacja parametrów:** obraz Wignera  $\rightarrow$  parametry  $(n, \alpha, p)$ .

## 11.5. Optymalizacja wydajności

Zapamiętywanie (cache) wyników kosztownych obliczeń operatorów przesunięcia w GKP za pomocą dekoratora `@lru_cache` (mechanizm Pythona, który automatycznie zapamiętuje wyniki wywołań funkcji i zwraca je bez ponownego obliczania). Zamiana pętli na operacje tablicowe NumPy (wektoryzacja — szybsze niż pętla `for` w Pythonie). Równoległe generowanie obrazów za pomocą modułu `multiprocessing`.

## 12. Harmonogram i zależności między fazami



Rysunek 1: Zależności między fazami planu działań. Linia przerywana = zależność częściowa.

**Zależności:**

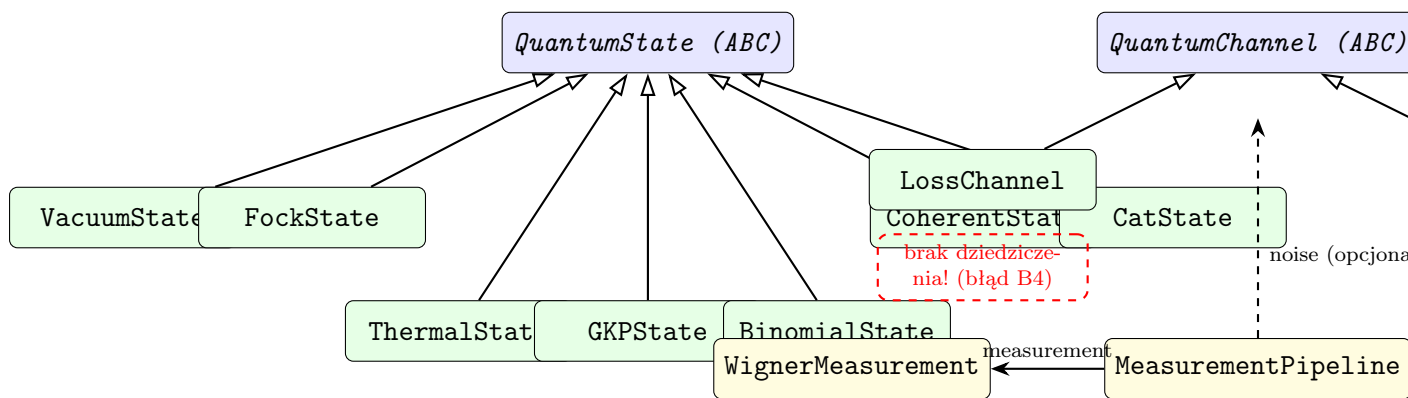
- Faza 2 wymaga ukończenia Fazy 1 (pakiety muszą działać przed konfiguracją CLI).
- Faza 4 wymaga ukończenia Fazy 1 i częściowo Fazy 3 (walidacja potrzebna do testów).



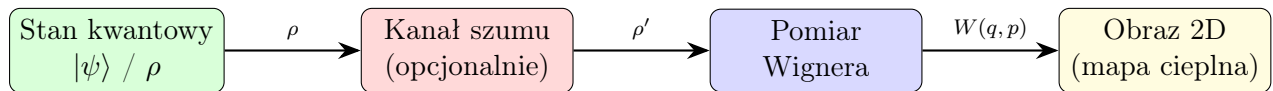
- Faza 3 i 4 mogą być realizowane częściowo równolegle.
- Faza 5 wymaga stabilnej bazy z Faz 1–4.

**Łączny szacowany nakład:** ok. 30 godzin roboczych (Fazy 1–4) + otwarte (Faza 5, głównie moduł ML).

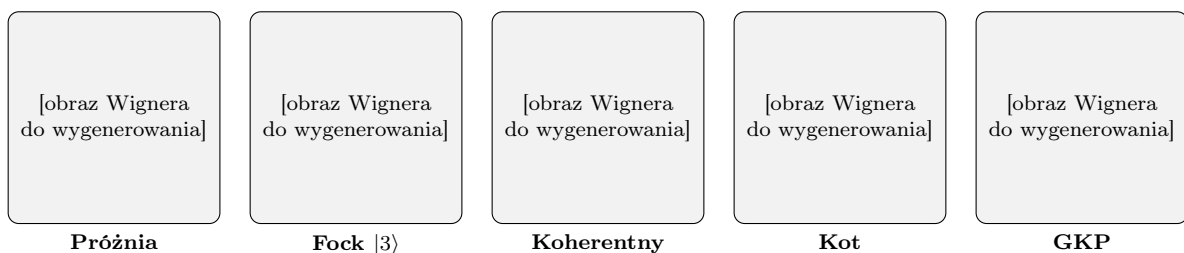
### 13. Diagramy architektury



Rysunek 2: Diagram klas projektu. Wszystkie stany dziedziczą bezpośrednio po `QuantumState`. Klasy abstrakcyjne (ABC) na niebiesko, konkretne na zielono. Czerwona ramka oznacza wykryty błąd B4.



Rysunek 3: Potok pomiarowy (pipeline): stan kwantowy  $\rightarrow$  szum  $\rightarrow$  pomiar Wignera  $\rightarrow$  obraz.



Rysunek 4: Przykładowe funkcje Wignera dla różnych stanów kwantowych (do wygenerowania z kodu projektu). Każdy stan ma charakterystyczny, unikalny wzór — to jest cecha, którą moduł ML będzie klasyfikował.

## 14. Podsumowanie

Projekt ma **solidne fundamenty architektoniczne** — przejrzysty podział na moduły, poprawne użycie klas abstrakcyjnych i trafny dobór bibliotek. Większość implementacji fizycznych jest poprawna.

Główne braki to: **brak struktury pakietu** (`__init__.py`), **brak testów**, **brak dokumentacji** oraz **kilka błędów matematycznych** do zweryfikowania. Brakuje też części ML, która jest kluczowym celem projektu (klasyfikacja i rekonstrukcja stanów).

**Zalecana kolejność prac:** Faza 1 → Faza 2 → Fazy 3 i 4 równolegle → Faza 5 (patrz diagram zależności na Rysunku 1).

**Szacowany łączny nakład pracy na Fazy 1–4:** ok. 30 godzin roboczych.

## Bibliografia

- [1] Göran Lindblad. „On the generators of quantum dynamical semigroups”. W: *Communications in Mathematical Physics* 48.2 (1976), s. 119–130. DOI: [10.1007/BF01608499](https://doi.org/10.1007/BF01608499). URL: <https://doi.org/10.1007/BF01608499>.
- [2] J. R. Johansson, P. D. Nation i Franco Nori. „QuTiP 2: A Python framework for the dynamics of open quantum systems”. W: *Computer Physics Communications* 184.4 (2013), s. 1234–1240. DOI: [10.1016/j.cpc.2012.11.019](https://doi.org/10.1016/j.cpc.2012.11.019). URL: <https://doi.org/10.1016/j.cpc.2012.11.019>.
- [3] Marios H. Michael i in. „New Class of Quantum Error-Correcting Codes for a Bosonic Mode”. W: *Physical Review X* 6.3 (2016), s. 031006. DOI: [10.1103/PhysRevX.6.031006](https://doi.org/10.1103/PhysRevX.6.031006). URL: <https://doi.org/10.1103/PhysRevX.6.031006>.
- [4] Daniel Gottesman, Alexei Kitaev i John Preskill. „Encoding a qubit in an oscillator”. W: *Physical Review A* 64.1 (2001), s. 012310. DOI: [10.1103/PhysRevA.64.012310](https://doi.org/10.1103/PhysRevA.64.012310). URL: <https://doi.org/10.1103/PhysRevA.64.012310>.
- [5] Howard M. Wiseman i Gerard J. Milburn. *Quantum Measurement and Control*. Cambridge University Press, 2009. ISBN: 978-0521804424. DOI: [10.1017/CB09780511813948](https://doi.org/10.1017/CB09780511813948). URL: <https://doi.org/10.1017/CB09780511813948>.